Interrupt or Wait?
A Systematic Study of Interruption in Coding Agents

Ziyang Chen zy7chen@umd.edu
Ishaan Kalra ishaank@umd.edu
Sihong Wu shwu@umd.edu
Haoyi You yuriiyou@umd.edu

1 Introduction

Current LLM coding agents operate on a turn by turn basis. Once the model receives a user prompt, the LLM will start to work by interacting with the coding harness and the computer system over multiple turns until the agent finally confirm that the user's initial requirements are all fulfilled. This kind of setup falls short in cases where a user might have evolving needs. Currently, some harness accept interruption during the middle of the interaction between LLMs and the computer system by steering the user request into the prompts before the next LLM's inference. However, the impact of this mechanism on the coding agent system have not been clearly studied. It is unclear whether interruption during the execution will affect the outcomes or not, compared with waiting the agent to finish its current instructions. Moreover, it is also unclear how and when the behavior of agents will vary under the two different interaction protocols.

Possible hypotheses on the difference may include: Interruption may prevent obsolete work and reduce completion time, while waiting may produce useful components that make subsequent revisions easier.

To systematically examine the above problem, in this proposal, we first propose an interactive coding benchmark, in which the agent starts with an unspecified request and receives new requirements or revisions during implementation. The idea is to keep the initial request and codebase unchanged, while making changes based on subsequent user input. In this proposed coding benchmark, we will compare two interaction protocols: interruption and waiting.

We hope our research will serve both as a benchmark for evaluating coding agents and a systematic study on the following three questions:

1. Does different ways of delivering instructions affect outcomes?
2. What are the difference of the agent behavior in different cases?
3. When do those differences arise in coding agents?

2 Related Work

The main motivation to pursue this investigation comes from the new Interaction Models by Thinking Machines Lab (Lab, 2026). These models aim to provide a more collaborative interface to human-agent work, where they receive new input (stream of information) while responding or performing background work. This setting is related to prior work by Schlangen and Skantze on incremental dialogue processing (Schlangen and Skantze, 2009). Based on further exploring we came across a blog by Rajan Agarwal (Agarwal, 2026), where he tried to build his own tiny interaction models inspired by Thinking Machines Lab. In the blog, he explores a text-based setting in which an assistant follows typing, initiates lookups, and decides whether to integrate or skip returned information. LlamaPIE (Chen et al., 2025) similarly studies when proactive assistance is useful, redundant, or poorly timed. These works motivate us to investigate how incoming streams of user input (text, here) change ongoing coding work.

Benchmarks like InterruptBench (Zou et al., 2026), study how browser agents react when users add, revise, or retract requirements during execution. SID-Bench (Xia et al., 2026) is used to check whether an agent can distinguish between interruptions and transient confirmations, while ECHO (Zhao et al., 2026) is used to test whether an agent can respond differently to the same sentence based on context.

Recent work has also explored interactions in coding tasks. SWE-Together (Wu et al., 2026) uses tasks reconstructed from real coding sessions and a reactive user simulator to provide feedback during execution. SWE-Interact (Raghavendra et al., 2026) starts with incomplete instructions and gradually introduces requirements and revisions as the simulated user reviews the agent's work. MTAC-IFBench (Wen et al., 2026) investigates whether coding agents adhere to functional requirements and process constraints across multiple development rounds. FixedBench (Gloaguen et al., 2026) examines whether agents avoid unnecessary code changes when reported issues are resolved. Our project builds upon these works to control when requirements are clarified or changed, and the extent of their impact on existing code. We will compare updates that arrive before and after dependent code is written and check whether the agent revises affected components without disrupting behaviors that should remain unchanged.

3 Proposed Approach

To tackle the three problems we questioned in the introduction, we first develop a benchmark for the tasks where allows users to provide rough idea first, and then add and revise the requirement afterward, especially testing whether an agent can adapt to users' requirements as it codes.

We will then evaluate agents on the benchmark under two interaction protocols: interruption and waiting, and we will design the qualitative and quantitative analysis on the following questions:

Do different ways of delivering instructions affect outcomes? In this part, we will specifically evaluate the final correctness and fulfillment of revised requirements of the tasks, meaning whether the changed instructions are achieved.

What are the difference of the agent behavior in different cases? In this part, we will inspect action traces and successive code snapshots for the working spaces to check whether different agent conduct obsolete work, unnecessary rebuilding and redirected implementation. We will also consider how much editing they make in different cases, together with token budget, tool calls, running time. Another study will be how the agent preserve the unchanged requirements and code modules.

When do those differences arise in coding agents? We will evaluate how the above different behavior happens when the tasks are different.

3.1 Existing Benchmarks

We already discuss the related benchmarks in Section 2, and several existing benchmarks for coding agents only consider multiple interactions between users and agents, but they do not consider the distinction between interruption and waiting during interaction between the agent and the user. InterruptBench(Zou et al., 2026) consider interruption in web browsing environment, which is different from coding. Compared with web browsing, we believe that coding is a better environment to study interruption, because agents perform multiple steps and produce files and code structures that can be exaluated and compared systematically.

We will evaluate agents powered by different LLMs, including open-weight and closed-weight models, such as DeepSeek-V4.1-Flash(Xu et al., 2026), Qwen3.6-27B(Qwen Team, 2026), GPT-6 Astra(OpenAI, nd), and Claude Opus 5.5(Anthropic, nd). We will also consider some other models, if necessary.

3.2 Computing Resources

In this project, we do not aim to train or fine-tune an LLM or any other neural networks, so no additional GPUs are required. We may need to host open-weight LLMs to do inference, so we plan to use some of the computing resources from UMIACS. *The key requirement of our study is adequate API resources to draw inference from online LLM providers. Currently, we do not have enough API credits that can help us finish this study, so we need supprt on API credits for finishing this project.*

4 Data

We will first try to build our current benchmark based on SlopCodeBench (Orlanski et al., 2026), a well known benchmark for coding agents, to avoid collecting dataset and multi-turn instructions ourselves from scratch and exhaust too much time to collect the initial dataset.

A probable problem for SlopCodeBench is that it may be too hard and thus it is hard to distinguish the problem we want to study in the real cases, so we will also prepare a fallback option such as SWE-Bench (Jimenez et al., 2024). In this case,

we will need to modify the current coding benchmark based on our requirement on interactive coding between the user and the agent. The major update will be changing the initial user prompt and split the prompt into small pieces of instructions and providing the instructions gradually as the agent performs software engineering tasks.

In this case, we can avoid spending times on collecting coding dataset, while we can still conduct study on coding agents that interact with users during execution. Our pilot study will only conduct research on a small fraction of the SlopCodeBench dataset, and we will extend it to the whole dataset, once we get enough preliminary result on pilot study.

5 Tools

We will build our benchmark on SlopCodeBench and use mini-SWE-agent as the agent harness. We will retain most of the existing harness infrastructure. The agents will use open-weight or closed-weight LLMs as their underlying models. We will use the *openai* for API calls and vLLM to serve open-weight models.

We will also use OpenAI Codex and Claude Code to help us coding assistance, brainstorming, and redteaming, but the submitted proposal is hand-written by the authors without AI generated content.

6 Schedule

We plan to divide our project into the following schedule:

1. Prepare the dataset and extend the harness (3 weeks): we will modify the coding agent benchmark we select to fit in the interactive setting. Also, we will select some specific tasks for us to conduct pilot study. We will extend the harness to record execution logs and snapshots of the code the agent edits, and to support both interruption and waiting.

2. Conduct the pilot study and prepare the detailed evaluation protocol (2 weeks): We will conduct our study on pilot dataset to avoid wasting computation resources. The main purpose of conducting pilot study is to design the detailed evaluation protocol, including what quantitative metrics and qualitative metrics should be considered in the main study when we vary from different interaction protocols in different evaluation problems.

3. Conduct the main study (2 weeks): Based on the information we gather in the pilot study, we will evaluate different LLMs across different tasks with repeated trials.

4. Adding additional quantitative and qualitative analysis (2 weeks): Based on the results we collect during the main research, we will provide some additional analysis, where we didn't take into consideration during the pilot study.

5. Work on final reports (1 week).

References

Agarwal, R. (2026). Tiny interaction models. rajan.sh. https://www.rajan.sh/tiny-interaction-models.

Anthropic (n.d.). Claude Opus 5.5. https://platform.claude.com/docs/en/models/opus-5-5/overview. Model documentation. Accessed October 9, 2026.

Chen, T., Batchelder, N. S., Liu, A., Smith, N. A., and Gollakota, S. (2025). LlamaPIE: Proactive in-ear conversation assistants. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T., editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13801–13824, Vienna, Austria. Association for Computational Linguistics.

Gloaguen, T., Mündler, N., Müller, M., Raychev, V., and Vechev, M. (2026). Coding agents don't know when to act.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. (2024). SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Lab, T. M. (2026). Interaction models: A scalable approach to human-ai collaboration. *Thinking Machines Lab: Connectionism*. https://thinkingmachines.ai/blog/interaction-models/.

OpenAI (n.d.). GPT-6 Astra. https://developers.openai.com/api/docs/models/gpt-6-astra. Model documentation. Accessed October 9, 2026.

Orlanski, G., Roy, D., Yun, A., Shin, C., Gu, A., Ge, A., Adila, D., Roberts, N., Sala, F., and Albarghouthi, A. (2026). Slopcodebench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*.

Qwen Team (2026). Qwen3.6-27B: Flagship-level coding in a 27B dense model. https://qwen.ai/blog?id=qwen3.6-27b. Accessed October 9, 2026.

Raghavendra, M., Gunjal, A., Sabharwal, A., and He, Y. (2026). Swe-interact: Reimagining swe benchmarks as user-driven long-horizon coding sessions.

Schlangen, D. and Skantze, G. (2009). A general, abstract model of incremental dialogue processing. In Lascarides, A., Gardent, C., and Nivre, J., editors, *Proceedings of the 12th Conference of the European Chapter of the ACL (EACL 2009)*, pages 710–718, Athens, Greece. Association for Computational Linguistics.

Wen, B., Wang, C., Gui, J., Zhang, H., Niu, Y., Ke, P., Yang, D., Wang, H., and Huang, M. (2026). Mtac-ifbench: Benchmarking instruction-following in multi-turn agentic coding.

Wu, Y., Zhao, Z., Li, S., Lee, H. H., Zhu, J., Wu, S., Yu, T., Li, S., Zhang, L., Fan, X., and Li, S. (2026). Swe-together: Evaluating coding agents in interactive user sessions.

Xia, K., Mu, B., Shi, X., Xu, J., and Xie, L. (2026). Semantic-aware interruption detection in spoken dialogue systems: Benchmark, metric, and model.

Xu, A., Li, B., Lin, B., Xue, B., Xian, B., Xu, B., Wu, B., Zhang, B., Deng, B., Yu, C., et al. (2026). Deepseek-v4. 1-flash: Pushing the limits of kv cache compression. *arXiv preprint arXiv:2609.19969*.

Zhao, S., Cai, H., Fan, W., Guo, Q., Wang, Z., Yang, D., Zhou, Z., Shang, Y., Wang, W., Yang, L., Zhou, S., and Song, Y. (2026). Same words, different actions: Paired turn-taking evaluation under rewritten dialogue contexts.

Zou, H. P., Miao, C., Huang, W.-C., Chen, Y., Zhou, Y., Zhang, H., Wu, Y., Fang, L., Gu, Z., Zhang, Z., Zheng, K., Wang, F., Nian, Y., Li, S., Fan, W., He, L., Zhang, W., Liu, X., and Yu, P. S. (2026). When users change their mind: Evaluating interruptible agents in long-horizon web navigation.